

Table and Data Structure

A NoSQL database through MongoDB is used to provide the database.

Below shows an example of the structure of the data. The data is unstructured

And stored in collections which are similar to rows and columns in relational databases. Below is a screenshot of how data is organized for the indicators.

```
2013: 31541209
2014: 32716210
2015: 33753499
2016: 34636207
2017: 35643418
2018: 36686784
2019: 37769499
2020: 38972230
2021: 40099462
2022: 41128771
_id: ObjectId('6530b1fdaa822965c0ccd817')
Country Name: "Afghanistaan"
Series Name: "Total Population"
```

```
2013: 4.822785479
2014: 0.566944541
2015: 2.447563002
2016: -2.197526466
2017: 2.403656188
2018: 2.071348579
2019: 6.521479845
2020: 7.821667127
2021: 0.47742893
2022: ".."
_id: ObjectId('6530b1fdaa822965c0ccd818')
Country Name: "Afghanistan"
Series Name: "Yearly Inflation"
```

```
_id: ObjectId('6530b5f597cc9fdb2369554')
country: "Abkhazia"
capital: "Sukhumi"
type: "countryCapital"
```

```
_id: ObjectId('6530b5f597cc9fdb2369555')
country: "Afghanistan"
capital: "Kabul"
type: "countryCapital"
```

```
_id: ObjectId('6530b5f597cc9fdb2369556')
country: "Akrotiri and Dhekelia"
capital: "Episkopi Cantonment"
type: "countryCapital"
```

```
_id: ObjectId('6530b5f597cc9fdb2369557')
country: "Albania"
capital: "Tirana"
type: "countryCapital"
```

```
_id: ObjectId('6530b5f597cc9fdb2369558')
country: "Algeria"
capital: "Algiers"
type: "countryCapital"
```

```
_id: ObjectId('6530b5f597cc9fdb2369559')
country: "American Samoa"
capital: "Pago Pago"
type: "countryCapital"
```

Visualization of the product

Figure 1- Main Dashboard Menu. The button opens up a tab menu to navigate to different menus

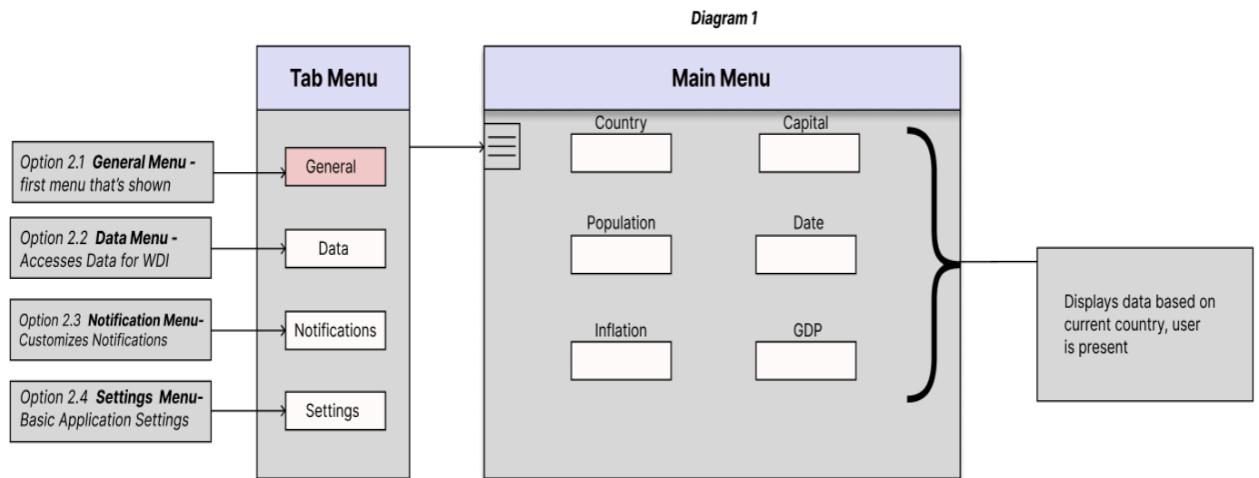


Figure 2- Economic Data Menu, allows user to generate table and graph

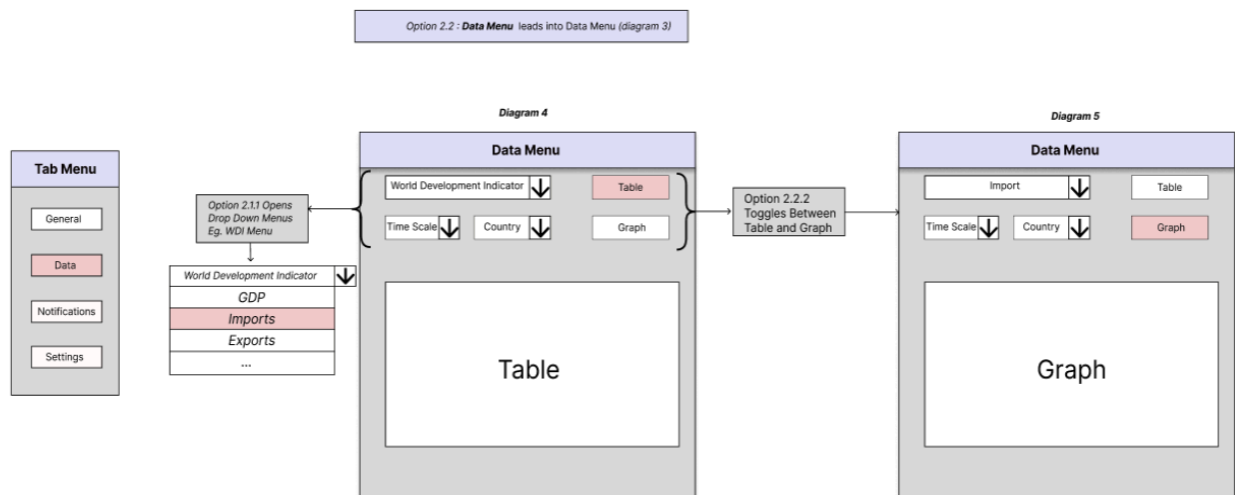


Figure 3 - Allows user to adjust notification settings

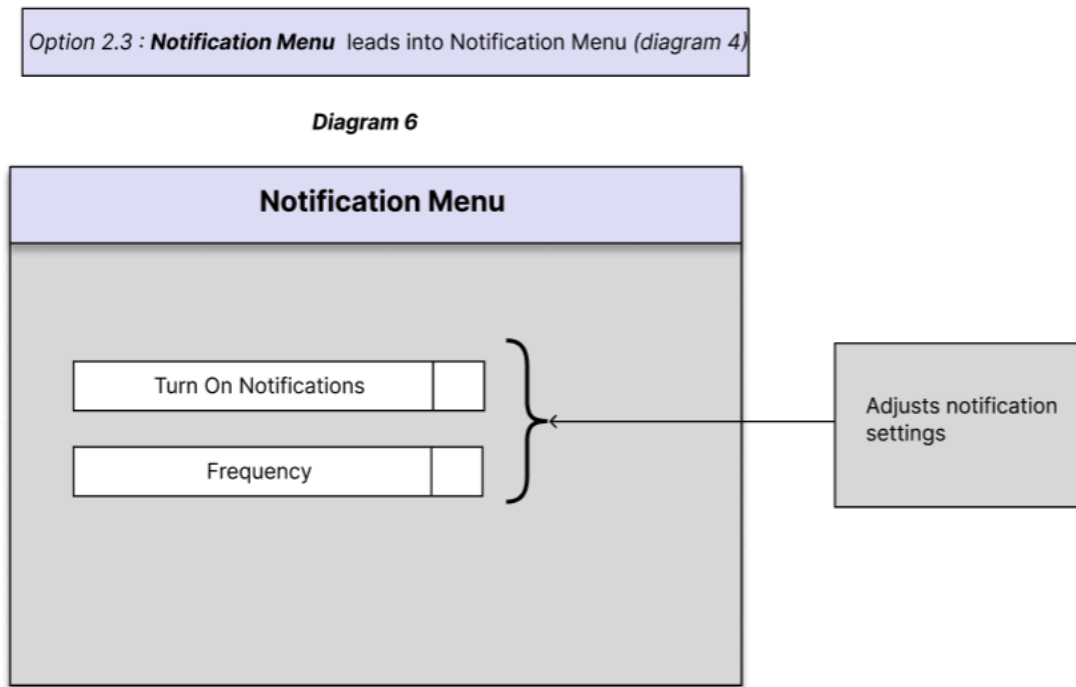
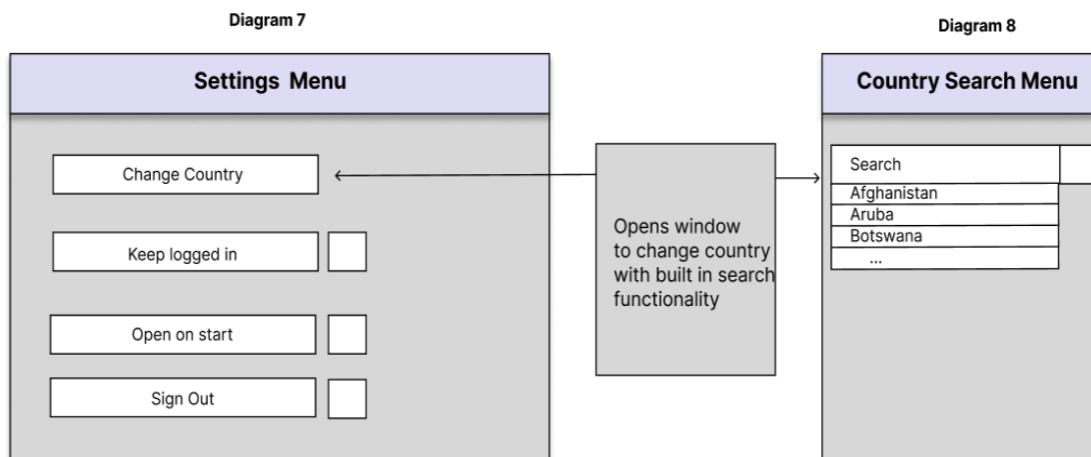
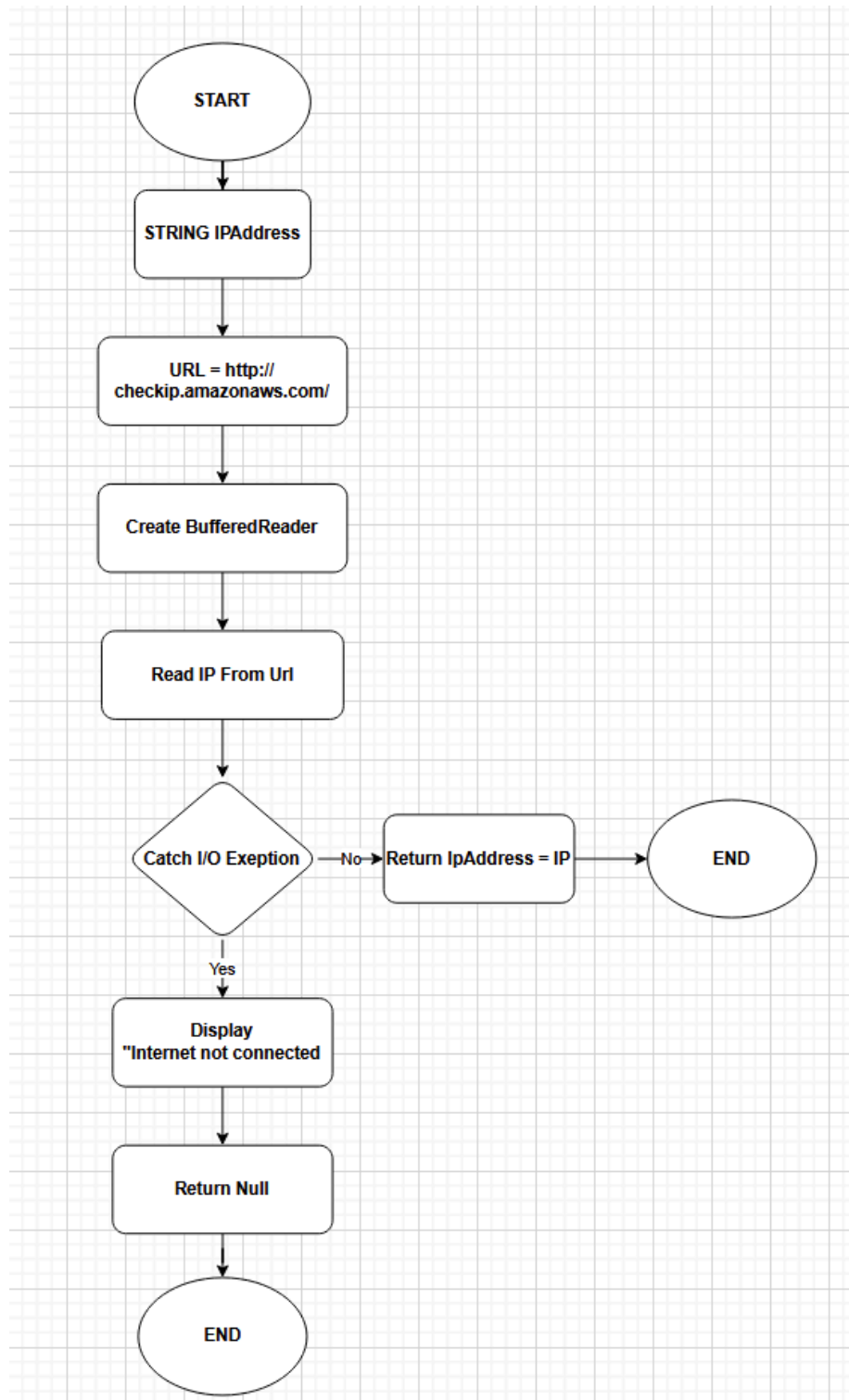


Figure 4 - Settings menu, allows user to change various settings about the program behaviour.

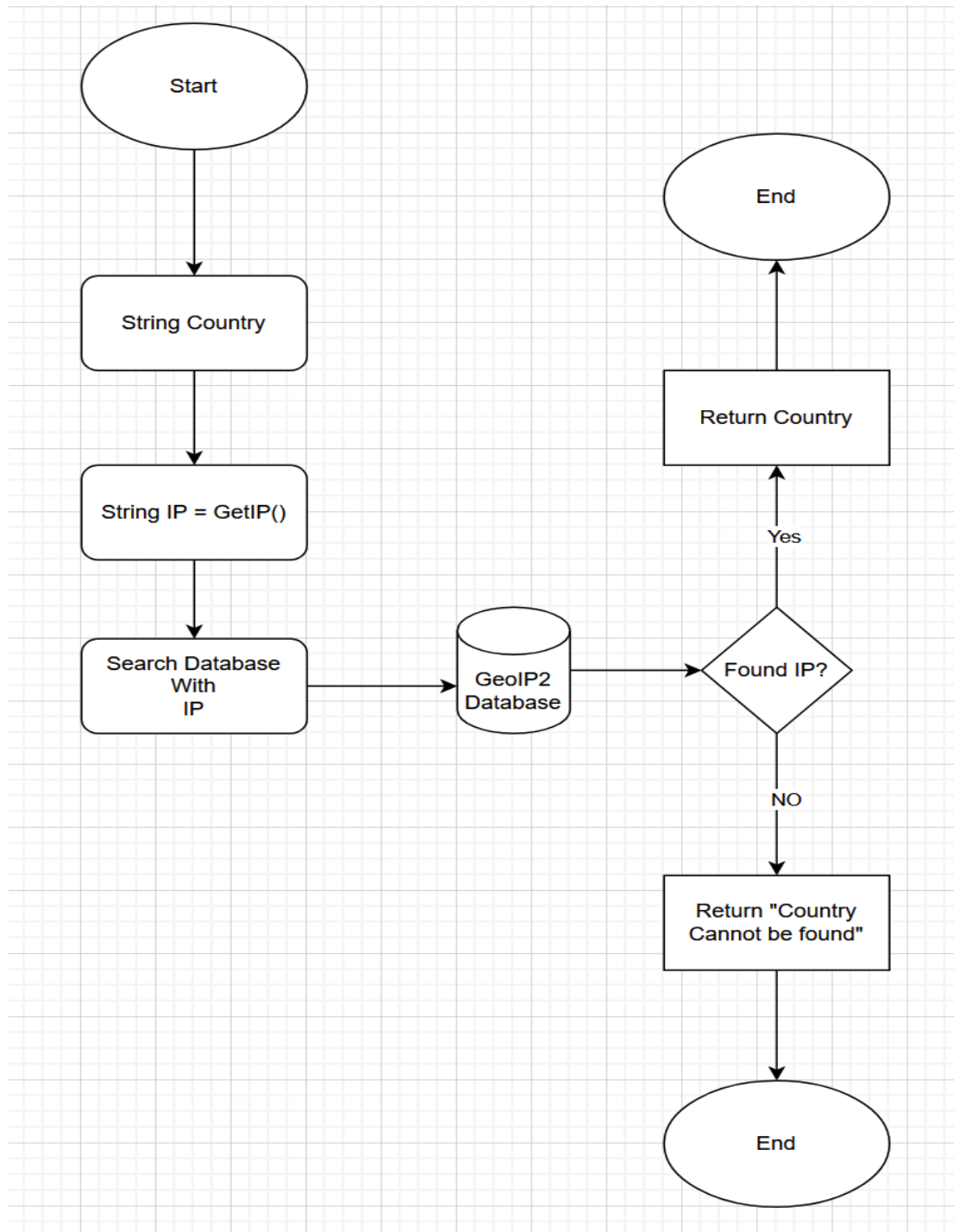


Flow Chart Of Algorithms

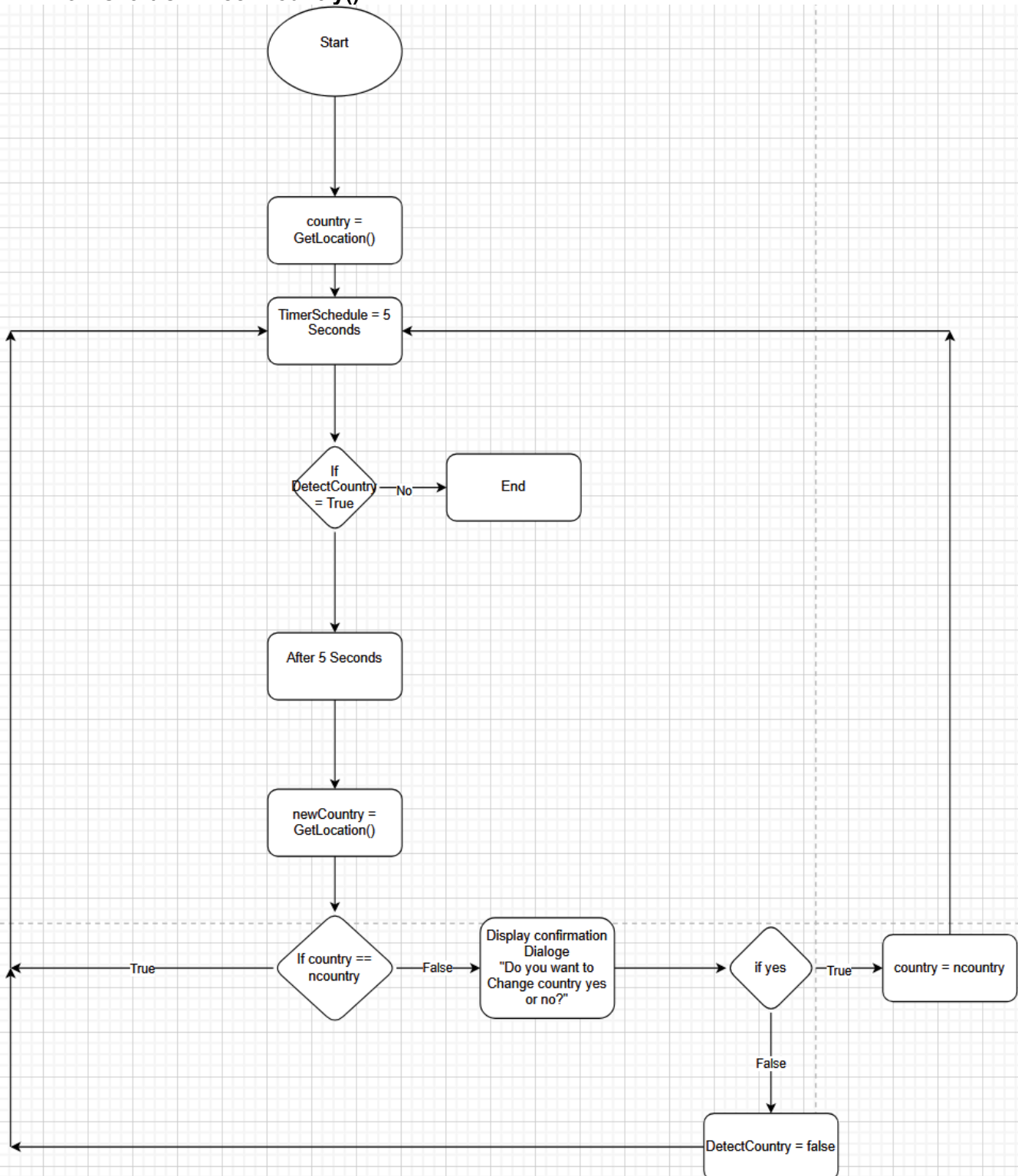
Flow Chart of **GetIP(**



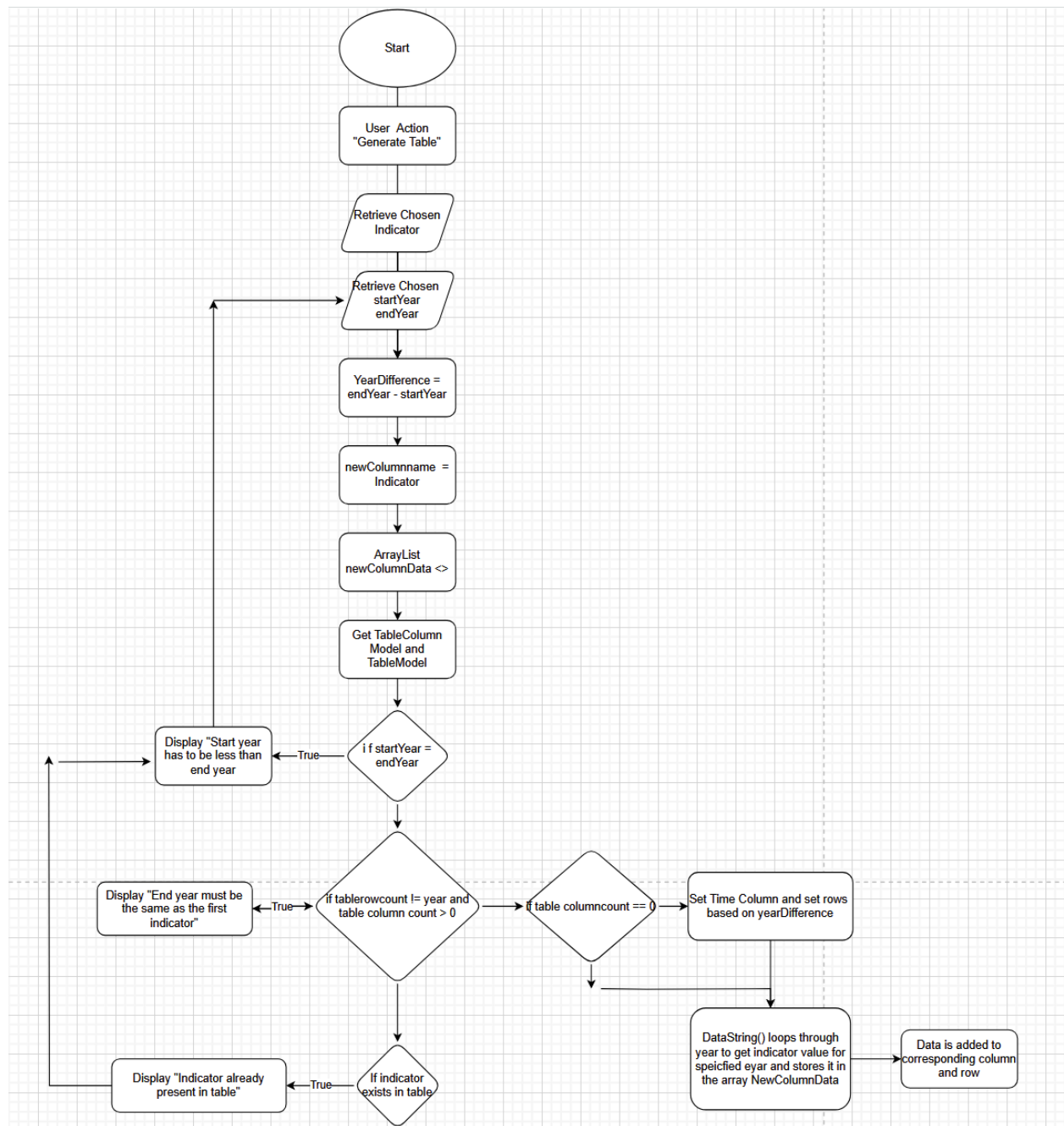
Flow Chart of **GetLocation()**



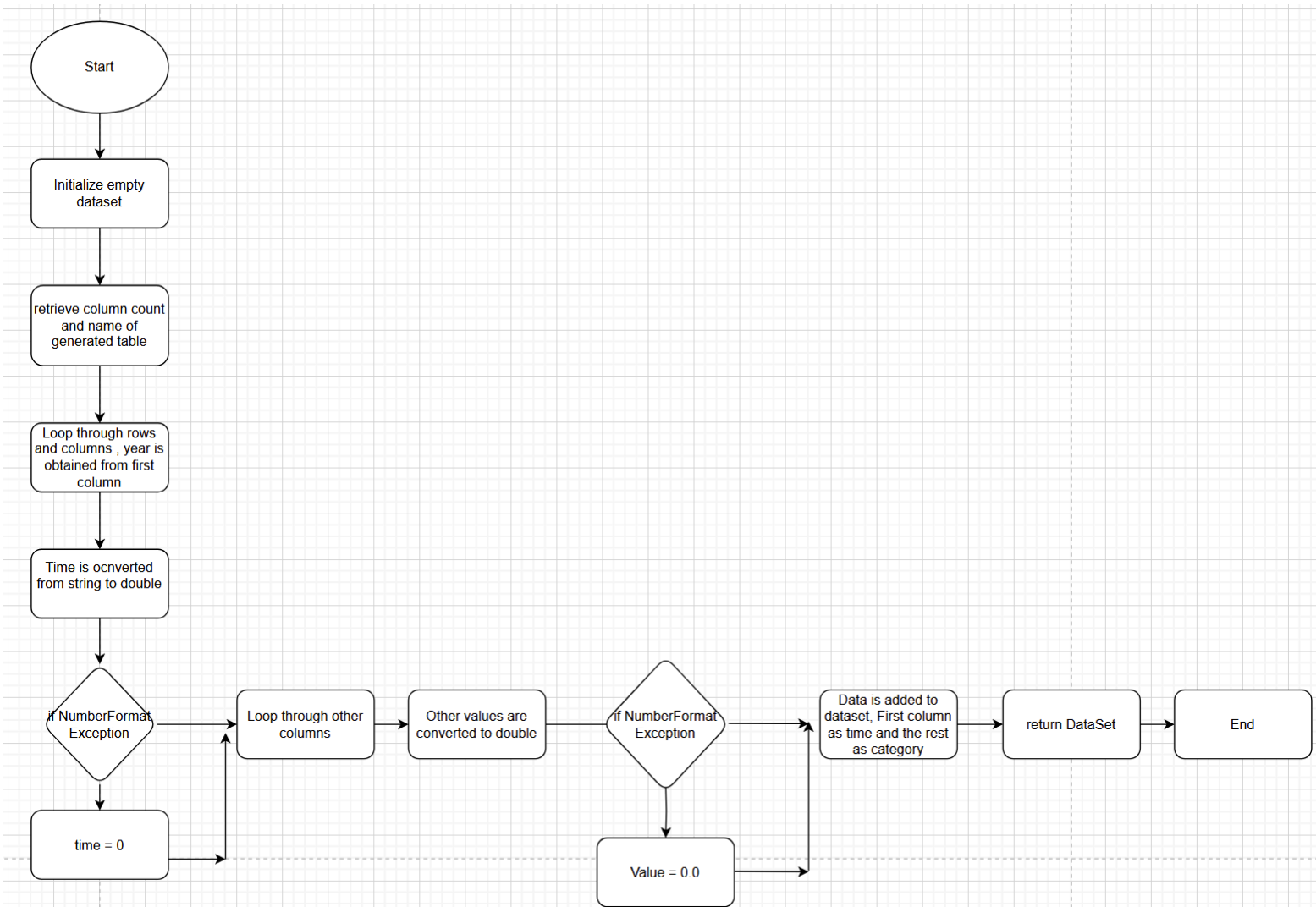
Flow Chart Of **CheckCountry()**



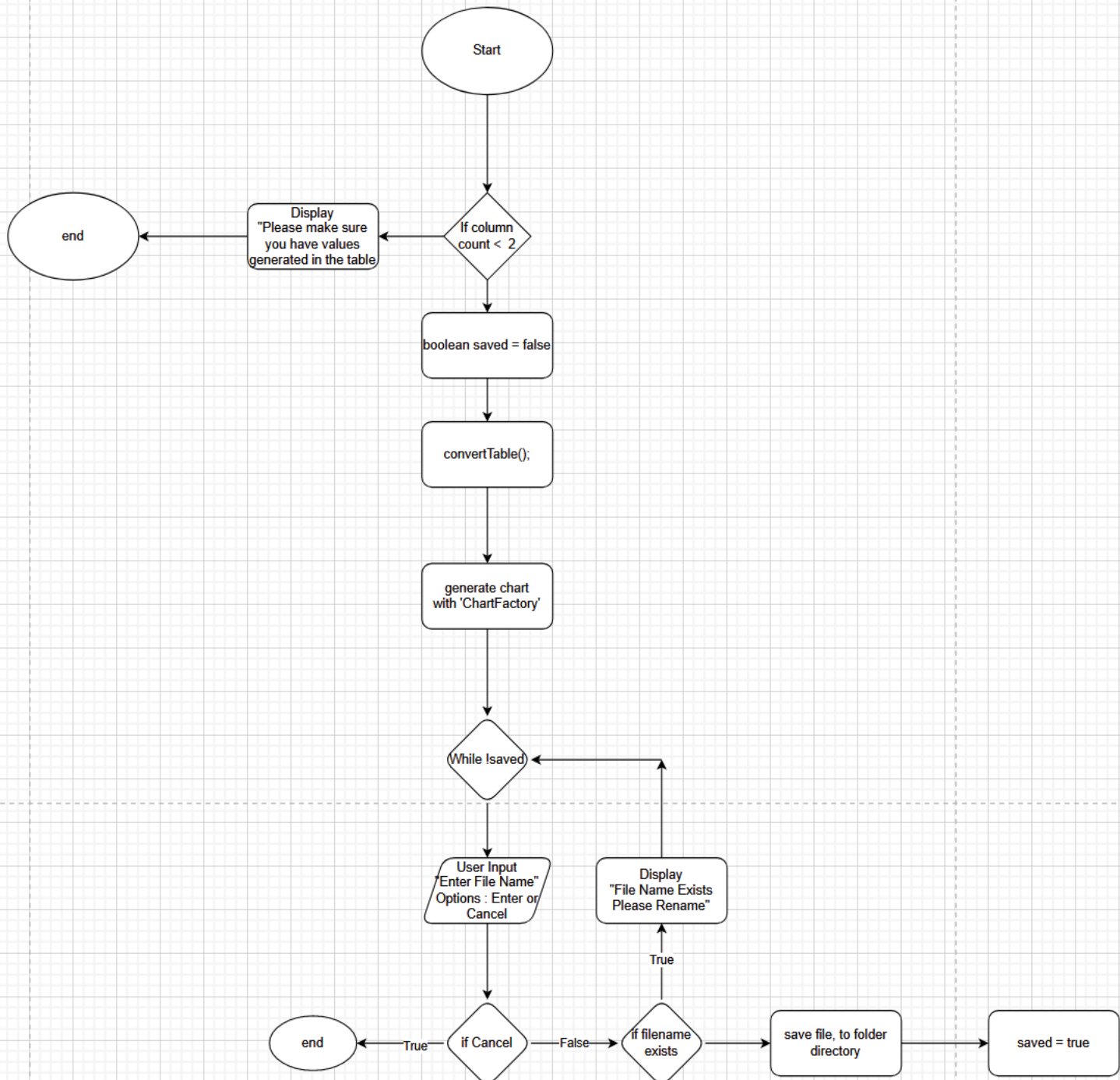
Flow Chart of Table Generation



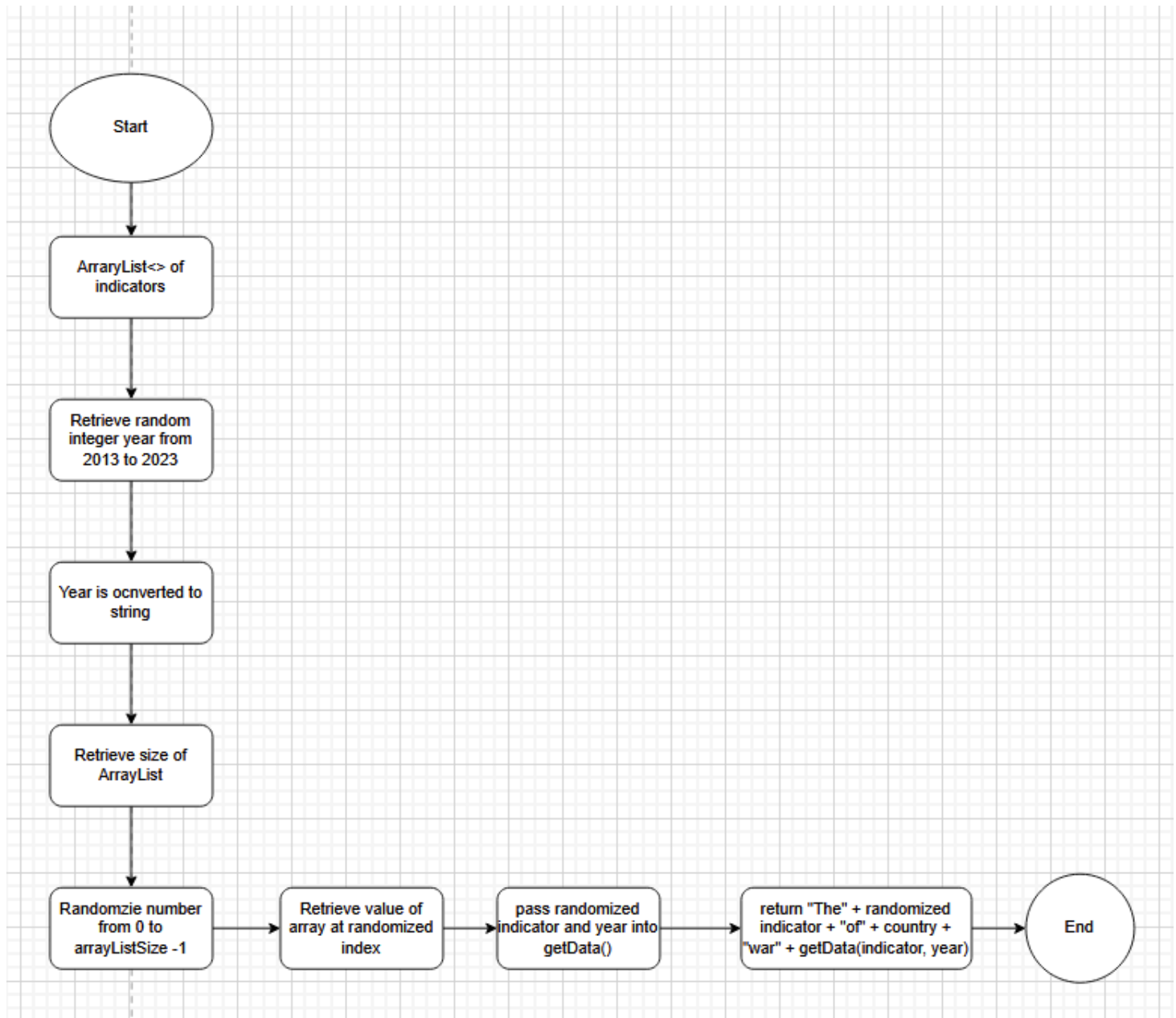
Flow Chart of **Converting Table to Chart Data Set. (convertTable())**



Flow Chart of **Saving Chart**



Flow Chart of **Fact Generation**



Pseudocode Of Algorithms

GetIp()

```
Class GetIP() {  
  
    String UrlString = "http://checkip.amazonaws.com/"  
    url = new URL(UrlString)  
    Output = Fetch Output with Bufferedreader  
    IpAddress = Output  
  
    If I/O Exception{  
        return "No Internet Connection"  
    }  
    End If  
    Else {  
        return ipAddress  
    }  
    End Else  
}
```

GetLocation()

```
Class GetLocation(){  
    Database = GeoIP2 Database  
    Address = GetIP()  
  
    If Address in Dataabase{
```

```

        AddressIndex = Address.getIndex()

        Country = valueCountAt(AddressIndex)

        return country;

    End If

    If I/O Exception{

        return "country can't be found"}

    End iF

    If GeoIP2 Exception{

        return "country can't be found"}

    End If

}

}

```

CheckCountry()

```

Class CheckCountry(){

    If detectcountry is true:

        TimerSchedule(period){

            If detectcountry is true:

                ncountry = GetLocation()

                If country is not equal to ncountry:

                    response = Dialouge "(Do you want to change?)"

```

End If

If response Yes

detectedcountry = true

country = GetLocation()

setText()

End If

Else:

detectcountry = false

detectedcountry = true

End Else

End if

End TimerSchedule

End if

}

DataString()

Class DataString(){

 output = GetData(collection, country, series, year)

 If output is an instance of Number:

 numericValue = CastToNumber(output)

 decimalFormat = CreateDecimalFormat("#,###.##")

 outputString = FormatNumberWithDecimalFormat(numericValue, decimalFormat)

 Return outputString

 End If

 If output is null:

 Return "N/A"

 End If

 If output is equal to "":

 Return "N/A"

 End If

}

TableGeneration

Input selectedStartYear

Input selectedEndYear

Input Indicator

yearstr

startyear = ParseInt(selectedStartYear)

endyear = ParseInt(selectedEndYear)

i = 0

yeardifference = endyear - startyear

newColumnData = CreateEmptyArrayList()

newColumnName = Indicator

Table = GetTableModel(jTable)

numberOfColumns = GetColumnCount(Table)

columnModel = GetColumnModel(Table)

if startyear > endyear:

 Display("End year must be greater than the start year!")

End If

if GetRowCount(Table) != yeardifference + 1 and GetColumnCount(tb1Model) > 0:

```

        Display("End Year Must be the Same, in order to add different indicators")
    End If

    if GetColumnCount(tb1Model) > 0:
        for I in Range(GetColumnCount(columnModel)):
            column = GetColumnAtIndex(columnModel, I)
            columnName = CastToString(GetHeaderValue(column))
            if newColumnName equals columnName:
                Display("Indicator Already Present In Table")
            End If
        End For
    End If

    Else{
        if numberOfColumns == 0:
            AddColumn(Table, "Time")
            selectedyear = endyear
            for rowIndex in Range(yeardifference + 1):
                AddRow(Table, ["", ""])
                newData = CastToString(endyear - i)
                SetValueAt(Table, newData, rowIndex, GetColumnCount(tb1Model) - 1)
                i = i + 1
            End For
        End If

        while startyear <= endyear:

```



```

        yearstr = CastToString(endyear)

        newData = DataString("Indicators", country, selectedIndicator, yearstr)

        AddToList(newColumnData, newData)

        endyear = endyear - 1

    End While

    AddColumn(Table, newColumnName)

    for rowIndex in Range(GetRowCount(Table)):

        newData = GetFromList(newColumnData, rowIndex)

        SetValueAt(Table, newData, rowIndex, GetColumnCount(tb1Model) - 1)

    End For

End If

hasduplicate = false

```

ConvertTable()

Class ConvertTable()

Create dataset

```

rowCount = GetRowCount(table)

```

```

columnCount = GetColumnCount(table)

```

```

for col in Range(1, columnCount):

```

```

    category = GetColumnName(table, col)

```

```

    for row in Range(rowCount):

```

```

        timeString = CastToString(GetValueAt(table, row, 0))

```

```
time = ParseDouble(timeString)
```

```
If NumberFormatException
```

```
    time = 0.0
```

```
End If
```

```
valueString = CastToString(GetValueAt(table, row, col))
```

```
value = ParseDouble(valueString)
```

```
If NumberFormatException
```

```
    value = 0.0
```

```
AddValueToDataset(dataset, value, category, CastToString(time))
```

```
Return dataset
```

Saving Chart

```
saved = false
```

```
while not saved:
```

```
    Input fileName = Display("Enter The Name You Would Like To Save The:")
```

```
    If fileName is null
```

Break

End If

file = CreateFile(path + "FileName")

If FileExists(file):

Input option = Display("Rename File")

If Yes:

Continue

Else:

Break

End If

Else:

SaveChartAsPNG

saved

End If

End If

End While

GenerateFact()

Class GenerateFact()

random = CreateRandomGenerator()

min = 2013

max = 2022

```
year = GenerateRandomNumberInRange(random, min, max)
```

```
yearstr = ConvertToString(year)
```

```
country = GetLocationData()
```

```
size = GetSizeOf(indicators)
```

```
indicatorIndex = GenerateRandomNumberInRange(random, 0, size - 1)
```

```
indicator = GetIndicatorAtIndex(indicators, indicatorIndex)
```

```
value = DataString("Indicators", country, indicator, yearstr)
```

```
fact = "Did you know that The " + indicator + " of " + GetLocationData() + " in the Year " +  
year + " was " + value
```

```
Return fact
```

Test Plan

Testing General Functionality(Dashboard)

Action to Test	Method Of Test	Expected Result
Detect Current Country	Open Up the application	It should automatically, set the country, capita, population and the other values to that which belongs to the country the user is currently in
Change Country	Change User's Current Country using a Combo Box	Country, Together with indicator values on the dashboard are changed
Detect New Country	Use a VPN, to connect to a different country location	A Popup message should appear, prompting the user that they are in a new country and whether they want to change. Country should change together with indicators present on the dashboard

Testing General Functionality(Economic Data)

Action to Test	Method Of Test	Expected Result
Generate Table	Click Generate Table Button	A Table with the selected Time Scale, Country and Indicator, should be generated, by getting data, with the GetData() function.
Generate Graph	Converts The Generated Table to a Graph, or Generates a Graph, by generating a table and converting it into a graph dataset	A graph, either line or bar, depending on the choice of the user should be generated

POTENTIAL ERROR HANDLING

Action to Test	Method Of Test	Expected Result
Generate Table but Country doesn't have data available	Click Generate Table Button	N/A is outputted onto the Table
Generate Graph, but the country doesn't have data available	Click Generate Graph Button	No gaps are left in a line chart, and it continues smoothly without breaks. For a bar chart, the given year data isn't available is empty.
Generate Table and Graph	No Time Scale/Indicator Is Selected	Error Message, provided to the user

Testing General Functionality(Notification Menu)

Action to Test	Method Of Test	Expected Result
Notification Text	Toggle Notifications and check to see if Null values in data aren't generated as notifications	All Null Values are skipped
Notification Timer	Start a timer, and see if notifications are sent after time passed	Notification is sent at a selected frequency.